

Математическая модель системы публикации со слотами, пастами и ограничениями

Техническое описание текущей реализации IdeaFlowBot

23 августа 2026 года

Аннотация

В документе формализована действующая система публикации IdeaFlowBot: недельные шаблоны слотов, построение автослотов, выбор времени с jitter, назначение контента, роль библиотеки паст, правила тегов и cooldown, защита от дубликатов, дневные лимиты и поведение publisher при ошибках Telegram. Система трактуется как жадный диспетчер, решающий локальную задачу удовлетворения ограничений для каждого очередного слота. Отдельно перечислены следствия, которые непосредственно вытекают из текущего кода, в том числе бессрочная блокировка точного повтора в одном канале и диагностический, а не резервирующий характер `paste_slots`.

Содержание

1	Назначение модели	4
2	Обозначения и сущности	4
3	Недельные шаблоны и экземпляры слотов	5
3.1	Шаблон слота	5
3.2	Связь слота с <code>publication log</code>	5
4	Автоматическое построение слотов	6
4.1	Количество готового живого контента	6
4.2	Сырой запрос на число слотов	6
4.3	Вместимость временного окна	7
4.4	Равномерное размещение	7
4.5	Применение плана к недельным шаблонам	8
5	Момент запуска и jitter	8
5.1	Окно обнаружения слота	8
5.2	Выбор фактической минуты	8

6	Ограничения канала	9
6.1	Локальный день и дневной максимум	9
6.2	Минимальное расстояние между публикациями	9
6.3	Рекламный blackout	9
7	Допустимость content item	10
7.1	Базовые условия	10
7.2	Лимиты по типу источника	10
7.3	Cooldown первичного тега	10
7.4	Cooldown шаблона	11
8	Порядок выбора кандидата	11
8.1	Этап 1: существующие nogenerated-items	11
8.2	Этап 2: паста из библиотеки	11
8.3	Этап 3: generated-items	12
9	Пасты как резервуар контента	12
9.1	Создание и преобразование	12
9.2	Область разрешённых каналов	12
9.3	Правила тегов	13
9.4	Источники сведений о последнем использовании	13
9.5	Глобальный и каналный cooldown пасты	14
9.6	Дополнительный cooldown канала на ту же пасту	14
9.7	Итоговый предикат доступности пасты	14
10	Нормализация и защита от дубликатов	15
10.1	Нормализация	15
10.2	Exact duplicate	15
10.3	Near duplicate	15
10.4	Следствие для повторного использования паст	16
11	Формальная задача назначения	16
12	Автоматы состояний	17
12.1	Content item	17
12.2	Publication log	17
13	Publisher и повторные попытки	18
13.1	Выбор due-публикации	18
13.2	Успех и постоянные ошибки	18
13.3	Экспоненциальный backoff	19

14 Числовые примеры	19
14.1 Расчёт автослотов	19
14.2 Ограничение вместимостью	20
14.3 Cooldown пасты	20
14.4 Jitter	20
15 Фактические свойства и ограничения текущей реализации	20
16 Карта параметров	21
17 Связь с исходным кодом	22
18 Заключение	22

1 Назначение модели

В системе необходимо разделять четыре разных объекта:

1. **слот** задаёт повторяющуюся временную позицию канала;
2. **паста** хранит повторно используемый текст и правила его применимости;
3. **content item** представляет конкретного кандидата на публикацию;
4. **publication log** связывает content item с конкретным экземпляром слота и хранит результат отправки.

Паста не публикуется напрямую. Полный путь имеет вид

$$\text{PasteLibrary} \longrightarrow \text{ContentItem}(\text{paste}) \longrightarrow \text{PublicationLog}(\text{slot}) \longrightarrow \text{Telegram}. \quad (1)$$

Аналогично сообщения пользователей, редакторские материалы и сгенерированные тексты перед публикацией приводятся к единой сущности `ContentItem`:

$$\{\text{Submission}, \text{Editorial}, \text{Generated}, \text{Paste}\} \longrightarrow \text{ContentItem}. \quad (2)$$

Таким образом, scheduler не публикует исходные сущности. Он работает с унифицированным множеством кандидатов и при необходимости сам создаёт кандидата из библиотечной пасты.

2 Обозначения и сущности

Пусть заданы следующие множества:

Обозначение	Смысл
C	множество каналов;
S_c	множество шаблонов слотов канала $c \in C$;
I_c	множество content items канала c ;
P	библиотека паст;
L	множество записей publication log;
B_c	множество рекламных blackout-интервалов канала c ;
$\mathcal{T}$	временная шкала UTC;
Z_c	временная зона канала c .

У content item имеется источник

$$q_i \in \{\text{submission}, \text{editorial}, \text{paste}, \text{generated}\} \quad (3)$$

и состояние

$$z_i \in \{\text{draft}, \text{pending_review}, \text{approved}, \text{scheduled}, \text{published}, \text{rejected}, \text{hold}\}. \quad (4)$$

Статус publication log принадлежит множеству

$$z_\ell \in \{\text{scheduled}, \text{sent}, \text{failed}, \text{cancelled}\}. \quad (5)$$

Под *живой публикацией в расписании* далее понимается log со статусом `scheduled` или `sent`. Именно эти два статуса учитываются в большинстве лимитов scheduler.

3 Недельные шаблоны и экземпляры слотов

3.1 Шаблон слота

Слот канала хранится как тройка

$$s = (c, w, \tau), \quad (6)$$

где c — канал, $w \in \{0, \dots, 6\}$ — номер дня недели, а τ — локальное время. В базе запрещены два шаблона с одинаковой комбинацией

$$(c, w, \tau). \quad (7)$$

Шаблон является недельным, а не одноразовым. Для локальной календарной даты d он порождает экземпляр слота тогда и только тогда, когда

$$\text{weekday}(d) = w. \quad (8)$$

Конкретный экземпляр обозначим

$$\sigma = (c, s, d). \quad (9)$$

Его номинальное локальное время и соответствующее время UTC:

$$t_\sigma^{\text{loc}} = \text{datetime}(d, \tau, Z_c), \quad (10)$$

$$t_\sigma^0 = \text{UTC}(t_\sigma^{\text{loc}}). \quad (11)$$

3.2 Связь слота с publication log

Введём бинарную переменную назначения

$$x_{i\sigma} = \begin{cases} 1, & \text{если item } i \text{ назначен в экземпляр слота } \sigma, \\ 0, & \text{иначе.} \end{cases} \quad (12)$$

Для каждого экземпляра действует ограничение

$$\sum_{i \in I_c} x_{i\sigma} \leq 1. \quad (13)$$

На уровне базы это закреплено уникальностью

$$(\text{channel_id}, \text{slot_id}, \text{slot_date}). \quad (14)$$

Важно, что ограничение базы распространяется на *все* статусы log. Поэтому даже failed или cancelled log уже занимает уникальную тройку. Логическая проверка scheduler ищет только scheduled/sent, но повторная вставка после failed/cancelled всё равно встретит ограничение уникальности и будет отменена через обработку ошибки целостности.

4 Автоматическое построение слотов

4.1 Количество готового живого контента

Для канала c и локального дня d обозначим его границы UTC через

$$[D_c^-(d), D_c^+(d)). \quad (15)$$

Индикатор готовности item i для этого дня:

$$\begin{aligned} R_i(c, d) = & \mathbf{1}[\text{channel}_i = c] \cdot \mathbf{1}[z_i = \text{approved}] \cdot \mathbf{1}[\text{scheduled_for}_i = \emptyset] \\ & \cdot \mathbf{1}[q_i \in \{\text{submission}, \text{editorial}\}] \\ & \cdot \mathbf{1}[\text{publish_after}_i = \emptyset \vee \text{publish_after}_i < D_c^+(d)] \\ & \cdot \mathbf{1}[\text{expires_at}_i = \emptyset \vee \text{expires_at}_i > D_c^-(d)]. \end{aligned} \quad (16)$$

Тогда число готовых живых материалов равно

$$A_c(d) = \sum_{i \in I_c} R_i(c, d). \quad (17)$$

При этом approved-пасты и generated-items в $A_c(d)$ не входят.

4.2 Сырой запрос на число слотов

Введём параметры канала:

$$\begin{aligned} P_c^{\max} &= \text{max_paste_per_day}, \\ M_c &= \text{max_posts_per_day}, \\ m_c &= \text{min_slots_per_day}. \end{aligned}$$

Пастовый fallback:

$$F_c = \begin{cases} P_c^{\max}, & \text{allow_pastes}_c = \text{true}, \\ 0, & \text{allow_pastes}_c = \text{false}. \end{cases} \quad (18)$$

Первичный запрос на слоты:

$$K_c^{\text{raw}}(d) = \max\{A_c(d), F_c, m_c\}. \quad (19)$$

После дневного лимита:

$$K_c^{\text{limit}}(d) = \min\{K_c^{\text{raw}}(d), \max(M_c, 0)\}. \quad (20)$$

4.3 Вместимость временного окна

Пусть a_c и b_c — начало и конец окна автослотов в минутах от начала локального дня, причём $b_c > a_c$. Длина окна

$$L_c = b_c - a_c. \quad (21)$$

Пусть

$$g_c = \max\{\text{min_gap_minutes}_c, 0\}. \quad (22)$$

Максимальное число минутных позиций в окне:

$$Q_c = \begin{cases} \left\lfloor \frac{L_c}{g_c} \right\rfloor + 1, & g_c > 0, \\ L_c + 1, & g_c = 0. \end{cases} \quad (23)$$

Итоговое число создаваемых времён:

$$K_c(d) = \min\{K_c^{\text{limit}}(d), Q_c\}. \quad (24)$$

Расчётное количество пастовых позиций:

$$K_c^{\text{paste}}(d) = \min\{\max(0, K_c(d) - A_c(d)), F_c\}. \quad (25)$$

Значение (25) сохраняется только в объекте результата планирования. Слоты не маркируются как пастовые, и scheduler не использует K_c^{paste} как квоту или резервирование.

4.4 Равномерное размещение

Если $K = 0$, список времён пуст. Если $K = 1$, единственный слот ставится в середину окна:

$$\tau_0 = a + \text{round}\left(\frac{L}{2}\right). \quad (26)$$

Если $K > 1$, времена вычисляются по формуле

$$\tau_j = a + \text{round}\left(\frac{jL}{K-1}\right), \quad j = 0, \dots, K-1. \quad (27)$$

Округление производится до целой минуты. Первый и последний слоты совпадают с границами окна, а промежуточные приближённо равноудалены.

4.5 Применение плана к недельным шаблонам

План строится не для отдельной таблицы датированных слотов. Он изменяет все шаблоны, соответствующие номеру дня недели целевой даты. Если параметр замены ручных слотов включён,

$$\text{auto_slots_replace_manual} = \text{true}, \quad (28)$$

planner может управлять и ручными шаблонами этого дня недели; иначе он удаляет или меняет только шаблоны с `is_auto_managed=true`.

Шаблоны, уже связанные в целевой день с log в состоянии `scheduled` или `sent`, не удаляются. Поле `auto_slots_last_planned_for` обеспечивает не более одного успешного применения плана к одной целевой дате.

5 Момент запуска и jitter

5.1 Окно обнаружения слота

Пусть T — текущее локальное время scheduler, t^0 — номинальное локальное время экземпляра слота, $J \geq 0$ — `slot_jitter_minutes`, а W — глобальный параметр `scheduler_window_minutes`. Слот рассматривается, если

$$t^0 - J \leq T \leq t^0 + \max(J, W). \quad (29)$$

На каждом запуске проверяются подходящие слоты локального вчера, сегодня и завтра. Это позволяет корректно захватывать экземпляры около границы UTC-дня.

5.2 Выбор фактической минуты

Пусть $\lceil T \rceil_{\min}$ означает округление вверх до целой минуты. При $J > 0$ строится минутная решётка

$$\mathcal{G}_\sigma(T) = \{t \in \mathcal{T}_{\text{minute}} : \max(t^0 - J, \lceil T \rceil_{\min}) \leq t \leq t^0 + J\}. \quad (30)$$

Из неё оставляются только моменты, для которых канал может публиковать:

$$\mathcal{F}_\sigma(T) = \{t \in \mathcal{G}_\sigma(T) : \text{CanPublish}(c, t) = 1\}. \quad (31)$$

Список случайно перемешивается, после чего берётся первый элемент. Если все моменты имеют одинаковую допустимость, это эквивалентно равномерному выбору на конечном множестве $\mathcal{F}_\sigma(T)$.

Если $J = 0$, система пытается использовать ровно t^0 . Если scheduler пришёл после $t^0 + J$, но ещё не позднее $t^0 + W$, допускается fallback

$$t = \lceil T \rceil_{\min}, \quad (32)$$

при условии $CanPublish(c, t) = 1$.

6 Ограничения канала

6.1 Локальный день и дневной максимум

Для момента t обозначим локальный день канала через $d_c(t)$, а его границы UTC через $[D_c^-(t), D_c^+(t))$. Число живых публикаций дня:

$$N_c(t) = \sum_{\ell \in L} \mathbf{1}[channel_\ell = c] \mathbf{1}[D_c^-(t) \leq scheduled_\ell < D_c^+(t)] \mathbf{1}[z_\ell \in \{\text{scheduled}, \text{sent}\}]. \quad (33)$$

Момент допустим только при

$$N_c(t) < M_c. \quad (34)$$

После назначения следующего item значение становится $N_c(t) + 1 \leq M_c$.

6.2 Минимальное расстояние между публикациями

Для каждого существующего живого log того же канала требуется

$$|t - scheduled_\ell| \geq g_c. \quad (35)$$

Код ищет публикации строго внутри интервала $(t - g_c, t + g_c)$, поэтому расстояние, в точности равное g_c , разрешено.

6.3 Рекламный blackout

Пусть

$$B_c = \bigcup_k [a_{ck}, b_{ck}) \quad (36)$$

— объединение рекламных blackout-интервалов. Scheduler требует

$$t \notin B_c. \quad (37)$$

Тем самым

$$CanPublish(c, t) = \mathbf{1}[t \notin B_c] \mathbf{1}[N_c(t) < M_c] \prod_{\ell \in L_c^{live}} \mathbf{1}[|t - scheduled_\ell| \geq g_c]. \quad (38)$$

Publisher повторно проверяет blackout уже в фактический момент отправки. Если отправка попала внутрь $[a, b)$, log не отменяется, а получает

$$retry_after = b. \quad (39)$$

7 Допустимость content item

7.1 Базовые условия

Item i может рассматриваться для времени t , если

$$z_i = \text{approved}, \quad (40)$$

$$publish_after_i = \emptyset \quad \vee \quad publish_after_i \leq t, \quad (41)$$

$$expires_at_i = \emptyset \quad \vee \quad expires_at_i > t. \quad (42)$$

Обозначим этот предикат через $Ready(i, t)$.

7.2 Лимиты по типу источника

Число паст, назначенных на локальный день:

$$N_c^{paste}(t) = \sum_{\ell \in L} \mathbf{1}[channel_\ell = c] \mathbf{1}[scheduled_\ell \in d_c(t)] \cdot \mathbf{1}[z_\ell \in \{\text{scheduled}, \text{sent}\}] \mathbf{1}[q_{item(\ell)} = \text{paste}]. \quad (43)$$

Для новой пасты необходимо

$$\text{allow_pastes}_c = \text{true}, \quad N_c^{paste}(t) < P_c^{\max}. \quad (44)$$

Аналогично для generated-контента:

$$\text{allow_generated}_c = \text{true}, \quad N_c^{gen}(t) < G_c^{\max}. \quad (45)$$

7.3 Cooldown первичного тега

Если у кандидата имеется $primary_tag_i = h$ и $H_c^{tag} = \text{same_tag_cooldown_hours}_c > 0$, определяется последняя успешная публикация того же primary tag:

$$u_{c,h} = \max\{published_\ell : channel_\ell = c, z_\ell = \text{sent}, primary_tag_{item(\ell)} = h\}. \quad (46)$$

Требуется

$$u_{c,h} < t - H_c^{tag}. \quad (47)$$

Запланированные, но ещё не отправленные публикации в этом cooldown не входят.

7.4 Cooldown шаблона

Для непастового item с $template_key_i = k$ вводится настройка

$$H_c^{template} = \text{same_template_cooldown_hours}_c > 0. \quad (48)$$

Тогда требуется

$$u_{c,k}^{template} < t - H_c^{template}, \quad (49)$$

где $u_{c,k}^{template}$ — момент последней успешной публикации того же шаблона. Для паст эта проверка намеренно не выполняется.

8 Порядок выбора кандидата

Алгоритм не решает глобальную задачу оптимизации по всем слотам. Он последовательно обходит каналы и due-slots и для каждого слота выполняет локальный жадный выбор.

8.1 Этап 1: существующие neregnerated-items

Для существующих кандидатов задаётся ранг источника

$$r(i) = \begin{cases} 0, & q_i = \text{submission}, \\ 1, & q_i = \text{editorial}, \\ 2, & q_i = \text{paste}, \\ 3, & q_i = \text{generated}, \\ 99, & \text{иначе.} \end{cases} \quad (50)$$

Сначала запрашиваются не более 50 approved-кандидатов, отличных от generated, в порядке

$$(r(i), \text{priority}_i, \text{created_at}_i) \uparrow. \quad (51)$$

Из этой последовательности выбирается первый item, прошедший лимиты и проверку дубликатов. Меньшее числовое значение `priority` означает более высокий приоритет; при равенстве раньше выбирается более старый item.

При отсутствии ограничения в 50 записей выбор можно было бы записать как

$$i^* = \arg \min_{i \in \mathcal{F}_{existing}(c,t)} (r(i), \text{priority}_i, \text{created_at}_i). \quad (52)$$

8.2 Этап 2: паста из библиотеки

Если существующего neregnerated-кандидата нет, scheduler запрашивает доступные пасты. Весь допустимый список случайно перемешивается, после чего рассматриваются первые

20 записей. Для каждой строится временный образ content item, на котором проверяются лимиты и дубликаты.

При успехе создаётся реальный item со значениями

$$q_i = \text{paste}, \quad (53)$$

$$\text{origin_paste_id}_i = \text{id}_p, \quad (54)$$

$$z_i = \text{approved}, \quad (55)$$

$$\text{review_required}_i = \text{false}. \quad (56)$$

Создание item и назначение в слот выполняются внутри одной вложенной транзакции.

8.3 Этап 3: generated-items

Только после неуспеха первых двух этапов выбирается существующий approved-item с $q_i = \text{generated}$. Для него также действуют временные ограничения, дневной лимит, cooldown по тегу и шаблону, а также проверка дубликатов.

Фактическая последовательность предпочтений:

$$\begin{aligned} & \text{submission} \succ \text{editorial} \\ & \succ \text{готовый paste item} \\ & \succ \text{новый item из PasteLibrary} \succ \text{generated}. \end{aligned} \quad (57)$$

9 Пасты как резервуар контента

9.1 Создание и преобразование

Паста может быть создана вручную, из submission либо из существующего content item. При создании сохраняются исходный текст, нормализованный текст, SHA-256 хеш, теги, primary tag, происхождение и cooldown-параметры.

Ручное преобразование пасты в content item по умолчанию создаёт

$$z_i = \text{pending_review}, \quad \text{review_required}_i = \text{true}. \quad (58)$$

Автоматическое преобразование scheduler, напротив, использует approved и $\text{review_required} = \text{false}$. Следовательно, факт нахождения пасты в активной библиотеке рассматривается автоматикой как достаточная редакционная санкция на её использование при выполнении остальных правил.

9.2 Область разрешённых каналов

Для пасты p и канала c :

$$\text{ScopeAllowed}(p, c) = \text{allow_all_channels}_p \vee \text{explicit_allow}(p, c). \quad (59)$$

Если $allow_all_channels_p = false$, должна существовать явная разрешающая пара (p, c) в таблице правил каналов пасты.

9.3 Правила тегов

Множество тегов пасты включает кэшированные теги и `primary tag`:

$$T_p = tags_p \cup \begin{cases} \{primary_tag_p\}, & primary_tag_p \neq \emptyset, \\ \emptyset, & primary_tag_p = \emptyset. \end{cases} \quad (60)$$

Пусть I_g и E_g — активные глобальные include/exclude-теги, а I_c и E_c — активные include/exclude-теги канала. Учитываются только правила, чей временной интервал содержит текущий момент.

Предикат теговой допустимости:

$$\begin{aligned} TagAllowed(p, c) = & \mathbf{1}[T_p \cap (E_g \cup E_c) = \emptyset] \\ & \cdot \mathbf{1}[I_g = \emptyset \vee T_p \cap I_g \neq \emptyset] \\ & \cdot \mathbf{1}[I_c = \emptyset \vee T_p \cap I_c \neq \emptyset]. \end{aligned} \quad (61)$$

Следствия формулы (61):

- любое пересечение с exclude немедленно запрещает пасту;
- при непустом глобальном include нужно совпадение с глобальным набором;
- при непустом канальном include независимо требуется совпадение с канальным набором;
- глобальный и канальный include соединены логическим “И”, а не “ИЛИ”.

9.4 Источники сведений о последнем использовании

Контекст доступности собирает три вида исторических отметок:

1. строки `PasteUsage`;
2. успешные `PublicationLog.published_at`;
3. импортированную историю канала с `matched_paste_id`.

Для каждой пасты берутся максимумы глобально и по каналу. Текущая функция объединения имеет следующую точную семантику. Пусть u — максимум `PasteUsage`, v — максимум `published-at`, h — максимум импортированной истории:

$$Combine(u, v, h) = \begin{cases} v, & u = \emptyset, \\ \max\{u, v, h\}, & u \neq \emptyset, \end{cases} \quad (62)$$

где отсутствующие аргументы не участвуют в максимуме. Поэтому при полном отсутствии `PasteUsage` импортированная история сама по себе не увеличивает последний момент использования; возвращается `published-at`. Это специальная совместимость с прежней семантикой текущего кода.

Отдельно вычисляется максимум `scheduled_for` для `log` со статусом `scheduled` или `sent`. Он называется последней резервацией и нужен, чтобы одна `scheduler`-сессия не назначила пасту повторно до фактической публикации.

9.5 Глобальный и каналный cooldown пасты

Эффективный глобальный cooldown ограничивается константой в три дня:

$$D_p^g = \min\{\max(global_cooldown_days_p, 0), 3\}. \quad (63)$$

Пусть U_p^g — последнее глобальное использование, R_p^g — последняя глобальная резервация, U_{pc} и R_{pc} — соответствующие значения в канале, а T — зафиксированный момент построения контекста. Тогда при $D_p^g > 0$ требуется

$$U_p^g < T - D_p^g, \quad (64)$$

$$R_p^g < T - D_p^g. \quad (65)$$

Для канального cooldown $D_p^c = per_channel_cooldown_days_p$ требуется

$$U_{pc} < T - D_p^c, \quad (66)$$

$$R_{pc} < T - D_p^c. \quad (67)$$

Если $D_p^c = 0$, прошлая отметка обычно не блокирует пасту, но резервация в настоящем или будущем всё равно удовлетворяет $R_{pc} \geq T$ и блокирует её.

После каждого нового назначения контекст немедленно обновляет R_p^g и R_{pc} . Поэтому глобальный cooldown может блокировать ту же пасту во всех остальных каналах уже в рамках текущего запуска scheduler.

9.6 Дополнительный cooldown канала на ту же пасту

В дополнение к собственному D_p^c пасты применяется каналный параметр

$$D_c^{same} = same_paste_cooldown_days_c. \quad (68)$$

Пусть S_{pc} — максимальное *scheduled_for* среди log той же пасты и того же канала в статусе *scheduled* либо *sent*. Для планируемого момента t требуется

$$S_{pc} < t - D_c^{same}. \quad (69)$$

Итак, канальная повторяемость пасты одновременно ограничена её собственным *per_channel_cooldown_days*, настройкой канала *same_paste_cooldown_days* и механизмом дубликатов.

9.7 Итоговый предикат доступности пасты

Без учёта проверки текста базовая доступность может быть записана как

$$\begin{aligned}
PasteEligible(p, c, T) = & \mathbf{1}[status_p = \text{active}] \mathbf{1}[\text{allow_pastes}_c] \\
& \cdot \mathbf{1}[ScopeAllowed(p, c)] \mathbf{1}[TagAllowed(p, c)] \\
& \cdot \mathbf{1}[GlobalCooldownAllowed(p, T)] \\
& \cdot \mathbf{1}[ChannelCooldownAllowed(p, c, T)].
\end{aligned} \tag{70}$$

После этого дополнительно применяются дневной лимит паст, каналный cooldown той же пасты, cooldown primary tag и проверка exact/near duplicate.

10 Нормализация и защита от дубликатов

10.1 Нормализация

Пусть $N(x)$ — последовательность преобразований текста:

1. очистка и схлопывание пробельных символов;
2. перевод в нижний регистр;
3. Unicode-нормализация NFKC;
4. удаление URL;
5. удаление упоминаний вида @name;
6. удаление пунктуации;
7. повторное схлопывание пробелов.

Точный идентификатор нормализованного текста:

$$H(x) = SHA256(N(x)). \tag{71}$$

Так как URL, mentions и пунктуация удаляются, тексты с различным исходным оформлением могут иметь одинаковый $H(x)$.

10.2 Exact duplicate

Кандидат i является точным дубликатом в канале c , если существует хотя бы одна успешная историческая публикация j с тем же хешем:

$$ExactDuplicate(i, c) = \mathbf{1}[\exists j : channel_j = c, z_{log(j)} = \text{sent}, H_i = H_j]. \tag{72}$$

У этого условия нет временного окна: поиск выполняется по всей доступной истории publication log.

10.3 Near duplicate

Пусть $Recent_{25}(c)$ — последние 25 успешно опубликованных items канала. Функция $sim(x, y) \in [0, 1]$ — коэффициент Python `SequenceMatcher.ratio` для нормализованных строк. При пороге $\theta = \text{similarity_threshold}$:

$$NearDuplicate(i, c) = \mathbf{1}\left[\max_{j \in Recent_{25}(c)} sim(N(text_i), N(text_j)) \geq \theta\right]. \tag{73}$$

По умолчанию $\theta = 0.82$. Итог:

$$Duplicate(i, c) = ExactDuplicate(i, c) \vee NearDuplicate(i, c). \quad (74)$$

Кандидат допускается только при $Duplicate(i, c) = 0$.

10.4 Следствие для повторного использования паст

Неизменённая паста всегда создаёт item с тем же нормализованным текстом и тем же хешем. После первой успешной публикации пасты p в канале c выполняется

$$ExactDuplicate(i_p, c) = 1 \quad (75)$$

для всех последующих items из той же неизменённой пасты. Поэтому истечение `same_paste_cooldown_days` само по себе не делает пасту повторно доступной в том же канале. На практике текущая схема позволяет безопасно распространять одну пасту по разным каналам, но запрещает точный повтор внутри одного канала бессрочно.

11 Формальная задача назначения

Для каждого due-slot σ scheduler фактически ищет пару (i, t) , такую что

$$t \in \mathcal{F}_\sigma(T), \quad (76)$$

$$Ready(i, t) = 1, \quad (77)$$

$$TypeLimits(i, c, t) = 1, \quad (78)$$

$$Cooldowns(i, c, t) = 1, \quad (79)$$

$$Duplicate(i, c) = 0. \quad (80)$$

Глобальная система ограничений может быть записана следующим образом:

$$\sum_i x_{i\sigma} \leq 1 \quad \forall \sigma, \quad (81)$$

$$\sum_\sigma x_{i\sigma} \leq 1 \quad \forall i \text{ в рамках нормального жизненного цикла}, \quad (82)$$

$$\sum_{i, \sigma: d_c(t_\sigma)=d} x_{i\sigma} \leq M_c \quad \forall c, d, \quad (83)$$

$$\sum_{i: q_i=\text{paste}} x_{i\sigma(d,c)} \leq P_c^{\max} \quad \forall c, d, \quad (84)$$

$$\sum_{i: q_i=\text{generated}} x_{i\sigma(d,c)} \leq G_c^{\max} \quad \forall c, d, \quad (85)$$

$$|t_\sigma - t_{\sigma'}| \geq g_c \quad \forall \sigma \neq \sigma' \text{ одного канала}, \quad (86)$$

$$t_\sigma \notin B_c \quad \forall \sigma. \quad (87)$$

Однако реализация не решает эту систему целиком и не максимизирует число заполненных слотов. Она фиксирует решения по одному слоту. Поэтому ранний жадный выбор потенциально может лишить более поздний слот кандидата, который подошёл бы ему лучше. Это нормальное свойство применяемого online-алгоритма.

Упрощённый псевдокод:

```
for channel in active_channels:
    for slot in due_slots(channel):
        if slot_is_used(slot):
            continue
        if not channel_can_publish(channel, slot.time):
            continue

        item = first_feasible_existing_non_generated()
        if item is None:
            item = create_from_random_feasible_library_paste()
        if item is None:
            item = first_feasible_generated()

        if item is not None:
            item.status = scheduled
            insert PublicationLog(slot, item, scheduled)
```

12 Автоматы состояний

12.1 Content item

Нормальный путь публикации:

$$\text{approved} \xrightarrow{\text{scheduler}} \text{scheduled} \xrightarrow{\text{Telegram success}} \text{published}. \quad (88)$$

При временной транспортной ошибке:

$$\text{scheduled} \xrightarrow{\text{transient error}} \text{scheduled}. \quad (89)$$

При постоянной ошибке данных или конфигурации:

$$\text{scheduled} \xrightarrow{\text{permanent error}} \text{approved}. \quad (90)$$

При неактивном канале `scheduled-item` также возвращается в `approved` и получает `scheduled_for = ∅`.

12.2 Publication log

Переходы log:

$$\text{scheduled} \xrightarrow{\text{success}} \text{sent}, \quad (91)$$

$$\text{scheduled} \xrightarrow{\text{transient}} \text{scheduled}, \quad (92)$$

$$\text{scheduled} \xrightarrow{\text{permanent}} \text{failed}, \quad (93)$$

$$\text{scheduled} \xrightarrow{\text{inactive channel}} \text{cancelled}. \quad (94)$$

После успешной публикации `item` из пасты создаётся `PasteUsage` с текущим моментом. Эта строка участвует в последующих `cooldown`-расчётах.

13 Publisher и повторные попытки

13.1 Выбор due-публикации

`Publisher` выбирает `log`, для которого

$$z_\ell = \text{scheduled}, \quad (95)$$

$$\text{scheduled}_\ell \leq T, \quad (96)$$

$$\text{retry_after}_\ell = \emptyset \quad \vee \quad \text{retry_after}_\ell \leq T. \quad (97)$$

Порядок выбора:

$$(\text{scheduled}_\ell, \text{id}_\ell) \uparrow. \quad (98)$$

За один шаг блокируется ровно одна строка через `FOR UPDATE SKIP LOCKED`. Это позволяет нескольким `worker`-процессам не брать одну и ту же публикацию и не удерживает весь `batch` из-за одного медленного Telegram-вызова.

13.2 Успех и постоянные ошибки

При успехе сохраняются Telegram message id и `published-at`, после чего

$$z_\ell = \text{sent}, \quad (99)$$

$$z_i = \text{published}, \quad (100)$$

$$\text{retry_after}_\ell = \emptyset. \quad (101)$$

Отсутствие `binding` саб-бота считается постоянной ошибкой:

$$z_\ell = \text{failed}, \quad (102)$$

$$z_i = \text{approved}. \quad (103)$$

Неактивный канал приводит к `cancelled`, а отсутствие самого `content item` или канала — к `failed log`.

13.3 Экспоненциальный backoff

Пусть $n \geq 1$ — номер уже начатой попытки, B — базовая задержка, $B_{\max}$ — максимум. Локальная задержка publisher:

$$b_n = \min \{ B \cdot 2^{\min(\max(n-1,0),4)}, B_{\max} \}. \quad (104)$$

Если Telegram передал собственное значение $r = \text{retry_after}$, итог:

$$\Delta_n = \max\{b_n, r\}. \quad (105)$$

Новая попытка разрешена после

$$\text{retry_after} = T + \Delta_n. \quad (106)$$

При значениях по умолчанию $B = 60$ секунд и $B_{\max} = 900$ секунд последовательность локальных задержек равна

$$60, 120, 240, 480, 900, 900, \dots \quad (107)$$

После первой временной транспортной ошибки текущий проход publisher прекращается. Это выполняет роль простого circuit breaker и предотвращает шторм запросов при общей недоступности Telegram.

14 Числовые примеры

14.1 Расчёт автослотов

Пусть канал имеет параметры

$$A = 2, \quad P^{\max} = 3, \quad m = 4, \quad M = 6, \quad (108)$$

окно 10:00–22:00 и $g = 60$ минут. Тогда

$$F = 3, \quad (109)$$

$$L = 720, \quad (110)$$

$$Q = \left\lfloor \frac{720}{60} \right\rfloor + 1 = 13, \quad (111)$$

$$K = \min\{\max(2, 3, 4), 6, 13\} = 4, \quad (112)$$

$$K^{\text{paste}} = \min\{\max(0, 4 - 2), 3\} = 2. \quad (113)$$

По формуле (27) получаем

$$10:00, \quad 14:00, \quad 18:00, \quad 22:00. \quad (114)$$

Ожидается два живых материала и до двух паст, но это не жёсткое распределение. Если живой item не пройдёт duplicate-проверку, его слот может занять паста. Если подходящих

паст нет, слот останется пустым. Если в очереди уже есть approved paste item, он участвует среди существующих кандидатов раньше библиотечного fallback.

14.2 Ограничение вместимостью

Пусть окно 10:00–12:00, запрошено $K^{limit} = 5$, а $g = 60$. Тогда

$$L = 120, \quad Q = \left\lfloor \frac{120}{60} \right\rfloor + 1 = 3, \quad (115)$$

поэтому реально создаются только три времени:

$$10:00, \quad 11:00, \quad 12:00. \quad (116)$$

14.3 Cooldown пасты

Пусть у пасты $D_p^g = 1$ день, $D_p^c = 120$ дней. Она использована в канале c_1 два часа назад. Тогда глобально

$$U_p^g \geq T - 1 \text{ день}, \quad (117)$$

поэтому паста временно недоступна даже каналу c_2 . После истечения глобального дня она может стать доступной в c_2 , но в c_1 останется заблокированной канальным cooldown и, после успешной публикации, exact-duplicate проверкой.

14.4 Jitter

Пусть номинальный слот равен 16:00, $J = 30$ минут, scheduler запущен в 15:42. Тогда минутная решётка до фильтрации:

$$\mathcal{G} = \{15:42, 15:43, \dots, 16:30\}. \quad (118)$$

Если из-за min-gap и blackout допустимы только

$$\mathcal{F} = \{15:55, 15:56, 16:20\}, \quad (119)$$

одна из этих трёх минут будет выбрана после случайного перемешивания.

15 Фактические свойства и ограничения текущей реализации

Ниже перечислены свойства, важные для интерпретации формул и настройки системы.

1. Слот не имеет типа контента. K^{paste} является диагностикой плана, а не резервом пастовых слотов.

2. Реальный контент имеет абсолютный, а не процентный приоритет. Параметр `prefer_real_ratio` хранится в модели и профилях, но текущий scheduler его не использует.
3. Параметр очереди пока декларативный. `min_ready_queue` не входит ни в формулу автослотов, ни в выбор кандидата.
4. Оценка активности пасты не применяется. `min_channel_activity_score` присутствует у пасты, но не участвует в построении контекста доступности.
5. Пасты выбираются случайно. После фильтрации доступный список перемешивается криптографическим генератором `SystemRandom`; явной оптимизации по “давно не использовалась” нет.
6. **Exact duplicate** бессрочен. Неизменённая паста после первой успешной публикации не сможет повториться в том же канале даже после окончания cooldown.
7. **Failed/cancelled log** фактически закрывает slot-day. Это следует из уникальности $(channel, slot, date)$ для всех статусов, несмотря на то что предварительная логическая проверка считает занятыми только `scheduled/sent`.
8. Автоплан меняет недельные шаблоны. Стандартное время планирования равно 23:30, а стандартное окно — 10:00–22:00. Поэтому новый набор времён создаётся после завершения текущего окна. Поэтому при первом запуске он не заполнит уже прошедший день, а как недельный шаблон практически повлияет на следующее наступление этого дня недели. В дальнейшем существовавший с прошлой недели шаблон может обслужить текущий день до вечернего перепланирования.
9. Алгоритм **online** и жадный. Нет глобального перераспределения кандидатов между будущими слотами и нет возврата к предыдущему выбору.

16 Карта параметров

Параметр	Математическая роль
<code>min_slots_per_day</code>	Нижняя граница запроса на число автослотов m_c .
<code>max_posts_per_day</code>	Верхняя граница числа <code>scheduled/sent</code> публикаций M_c .
<code>max_paste_per_day</code>	Дневной лимит паст $P_c^{\max}$ и пастовый fallback F_c .
<code>max_generated_per_day</code>	Дневной лимит <code>generated</code> -контента $G_c^{\max}$.
<code>min_gap_minutes</code>	Минимальное расстояние g_c между <code>scheduled-for</code> .
<code>slot_jitter_minutes</code>	Полуширина J интервала выбора фактической минуты.
<code>same_tag_cooldown_hours</code>	Cooldown H_c^{tag} по primary tag.
<code>same_template_cooldown_hours</code>	Cooldown H_c^{template} по шаблону непастового item.
<code>same_paste_cooldown_days</code>	Дополнительный каналный cooldown D_c^{same} пасты.
<code>global_cooldown_days</code>	Глобальный cooldown пасты, программно ограниченный тремя днями.
<code>per_channel_cooldown_days</code>	Собственный cooldown пасты в конкретном канале.
<code>allow_pastes</code>	Разрешает пастовый fallback и публикацию <code>paste-items</code> .
<code>allow_generated</code>	Разрешает <code>generated</code> -контент на последнем этапе выбора.
<code>similarity_threshold</code>	Порог θ near-duplicate проверки, по умолчанию 0.82.
<code>publisher_retry_base_seconds</code>	База экспоненциального backoff B , по умолчанию 60 секунд.

Параметр	Математическая роль
<code>publisher_retry_max_seconds</code>	Верхняя граница backoff $B_{\max}$, по умолчанию 900 секунд.

17 Связь с исходным кодом

Описание соответствует следующим компонентам проекта:

- модели каналов и слотов: `src/editorial/models/channel.py`;
- content items: `src/editorial/models/content.py`;
- пасты и история использования: `src/editorial/models/paste.py`;
- журнал публикаций: `src/editorial/models/publication.py`;
- автоматические слоты: `src/editorial/services/auto_slot_planner.py`;
- назначение контента: `src/editorial/services/scheduler.py`;
- доступность паст: `src/editorial/services/paste_service.py`;
- правила тегов: `src/editorial/services/tag_service.py`;
- отправка в Telegram: `src/editorial/services/publisher.py`;
- backoff и классификация ошибок: `src/editorial/services/telegram_resilience.py`;
- нормализация и similarity: `src/editorial/utils/text.py`.

18 Заключение

Система может быть охарактеризована как жадный стохастический scheduler с дискретным минутным временем. Автопланировщик определяет ёмкость недельного расписания по максимуму живой очереди, пастового fallback и минимального числа слотов, после чего ограничивает её дневным максимумом и геометрией временного окна. Основной scheduler для каждого due-slot случайно выбирает допустимую минуту и лексикографически предпочитает живой контент. Пасты образуют резервуар предварительно одобренного текста, но перед назначением проходят независимые каналные, теговые, cooldown- и duplicate-ограничения. Publisher завершает автомат состояний, обеспечивая идемпотентное владение due-log, экспоненциальные повторные попытки и возврат content item в approved при постоянной ошибке.